

שלב 1

תיאור הצעת הארגון:

הפרויקט יהיה בארגון בית קולנוע בבסיסי צה"ל

עיסוקו העיקרי של הקולנוע הצבאי הוא קיום הקרנות של סרטים לחיילים. חיילים יכולים לרכוש כרטיסים באמצעות מערכת פנימית ולצפות בסרטים.

בקולנוע ישנם מספר אולמות, כאשר בכל אולם מספר מושבים המחולקים לפי שורות. לכל אולם מספר שורות מסוים. בכל שורה 10 מושבים. השורות ממוספרות מ-1 והלאה והמושבים בכל שורה ממוספרים מ-1 עד 10.

כל מושב מאופיין באופן ייחודי במספר האולם, מספר השורה ומספרו בתוך השורה יחד.

בכל אולם ישנה הקרנה אחת בלבד בכל רגע נתון. ייתכן כי ביום אחד יוקרנו כמה הקרנות באותו אולם וכן יוקרנו כמה הקרנות במקביל באולמות שונים.

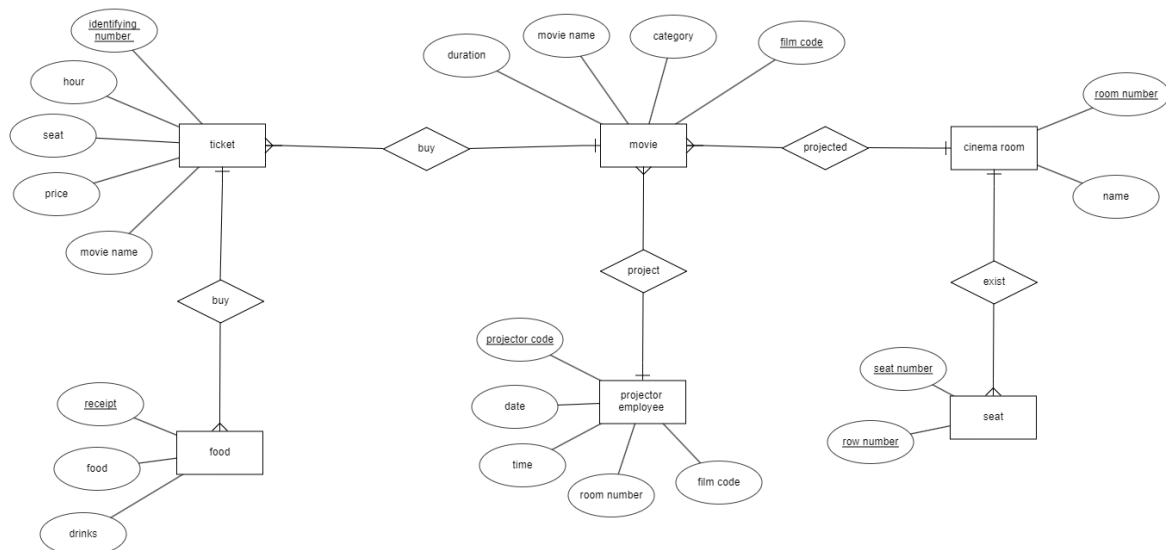
כל סרט יקוטלג בהתאם לתוכנו (דוקומנטרי, לילדים, דרמה, פשע ועוד).

בכל הקרנה מוקרן כמובן סרט אחד בלבד, אך סרט יכול בהחלט להיות מוקרן במספר הקרנות (בלי הגבלה).

כרטיס להקרנה הוא המקשר בין אוכל לבין הקרנה מסוימת ובין מושב. הוא מכיל את פרטי החייל, את המושב בו יישב, את מספר ההקרנה ואת המחיר.

חייבת להיות התאמה בין האולם בו ההקרנה מתקיימת לבין מספר המושב. כמו כן, לא ייתכן שבאותה ההקרנה יהיו שני כרטיסים בעלי אותו מספר מושב.

תרשים ERD



הטבלאות:

אולמות (מספר-אולם, שם-אולם)

מושבים (מספר-שורה, מספר-מושב)

סרטים (קוד-סרט, קטגוריה, שם-סרט, אורך)

עובד הקרנה (מספר-הקרנה, תאריך-הקרנה, שעת-הקרנה, מספר-אולם, קוד הסרט)

כרטיסים (מספר-כרטיס, שעה, מושב, מחיר, שם הסרט)

מזנון(קבלה, אוכל, שתייה)

פירוט הישויות:

אולם: לכל אולם מספר ייחודי והוא המפתח שלו. ומאפיין נוסף יהיה שם האולם.

מושב: לכל מושב מפתח ייחודי המורכב משלושה פרמטרים: מספר הכיסא מספר השורה, ומספר אולם.

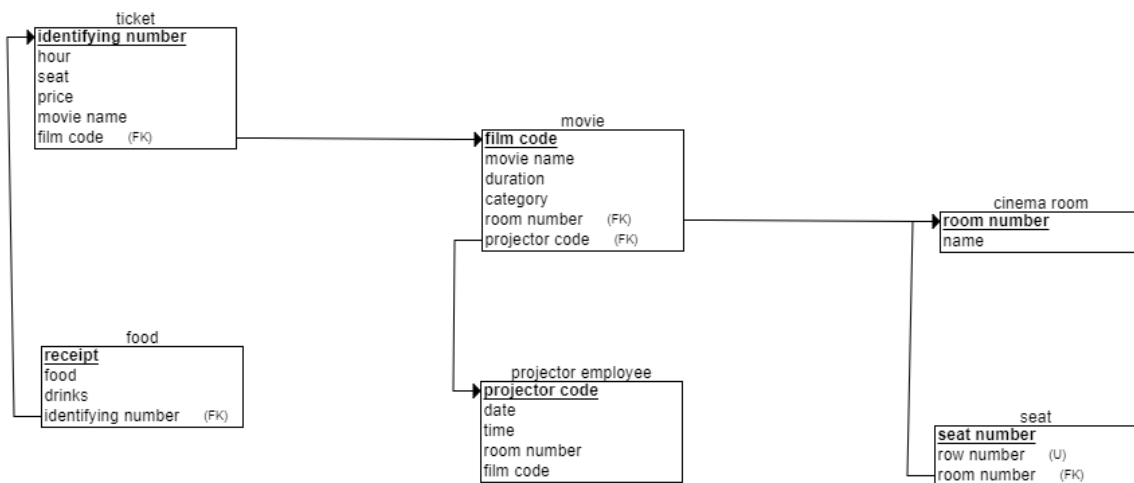
סרט: לכל סרט מספר ייחודי והוא המפתח שלו. ומאפיינים נוספים, קטגוריה, שם הסרט, ואורך הסרט, כך נוכל לחשב כמה זמן ההקרנה וכמה זמן האולם יהיה תפוס.

עובד הקרנה: יש לו מספר יחודי והוא המפתח שלו, תאריך, שעה, מספר חדר, קוד הסרט

כרטיס: מספר יחודי שהוא המפתח שלו, שעה, מיקום(מושב), מחיר, שם הסרט.

מזנון: קבלה ושזה יהיה המפתח, אוכל, ושתייה.

תרשים DSD:



:SQL

```

CREATE TABLE cinema_room
(
    room_number INT NOT NULL,
    name INT NOT NULL,
    PRIMARY KEY (room_number)
);

CREATE TABLE projector_employee
(
    projector_code INT NOT NULL,
    date INT NOT NULL,
    time INT NOT NULL,
    room_number INT NOT NULL,
    film_code INT NOT NULL,
    PRIMARY KEY (projector_code)
);

CREATE TABLE seat
(
    seat_number INT NOT NULL,
    row_number INT NOT NULL,
    room_number INT NOT NULL,
    PRIMARY KEY (seat_number),
    FOREIGN KEY (room_number) REFERENCES cinema_room(room_number),
    UNIQUE (row_number)
);

CREATE TABLE movie

```

```
(
    film_code INT NOT NULL,
    movie_name INT NOT NULL,
    duration INT NOT NULL,
    category INT NOT NULL,
    room_number INT NOT NULL,
    projector_code INT NOT NULL,
    PRIMARY KEY (film_code),
    FOREIGN KEY (room_number) REFERENCES cinema_room(room_number),
    FOREIGN KEY (projector_code) REFERENCES
projector_employee(projector_code)
);

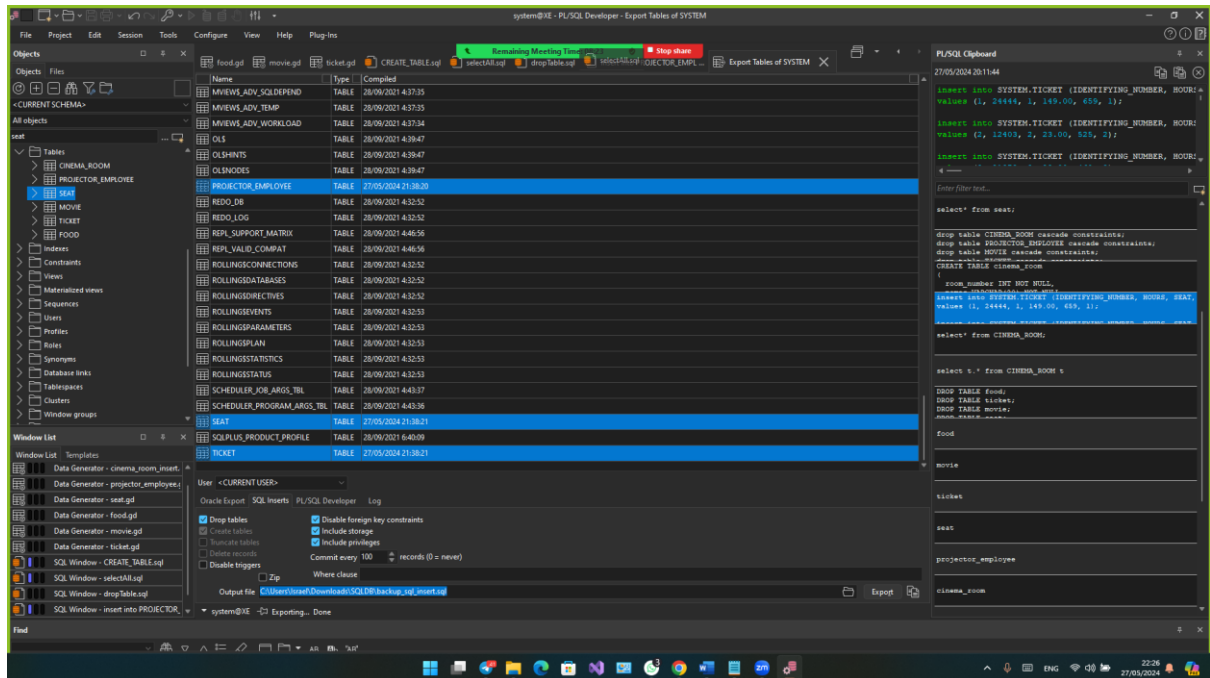
CREATE TABLE ticket
(
    identifying_number INT NOT NULL,
    hour INT NOT NULL,
    seat INT NOT NULL,
    price INT NOT NULL,
    movie_name INT NOT NULL,
    film_code INT NOT NULL,
    PRIMARY KEY (identifying_number),
    FOREIGN KEY (film_code) REFERENCES movie(film_code)
);

CREATE TABLE food
(
    food INT NOT NULL,
    drinks INT NOT NULL,
    receipt INT NOT NULL,
    identifying_number INT NOT NULL,
    PRIMARY KEY (receipt),
    FOREIGN KEY (identifying_number) REFERENCES
ticket(identifying_number)
);
```

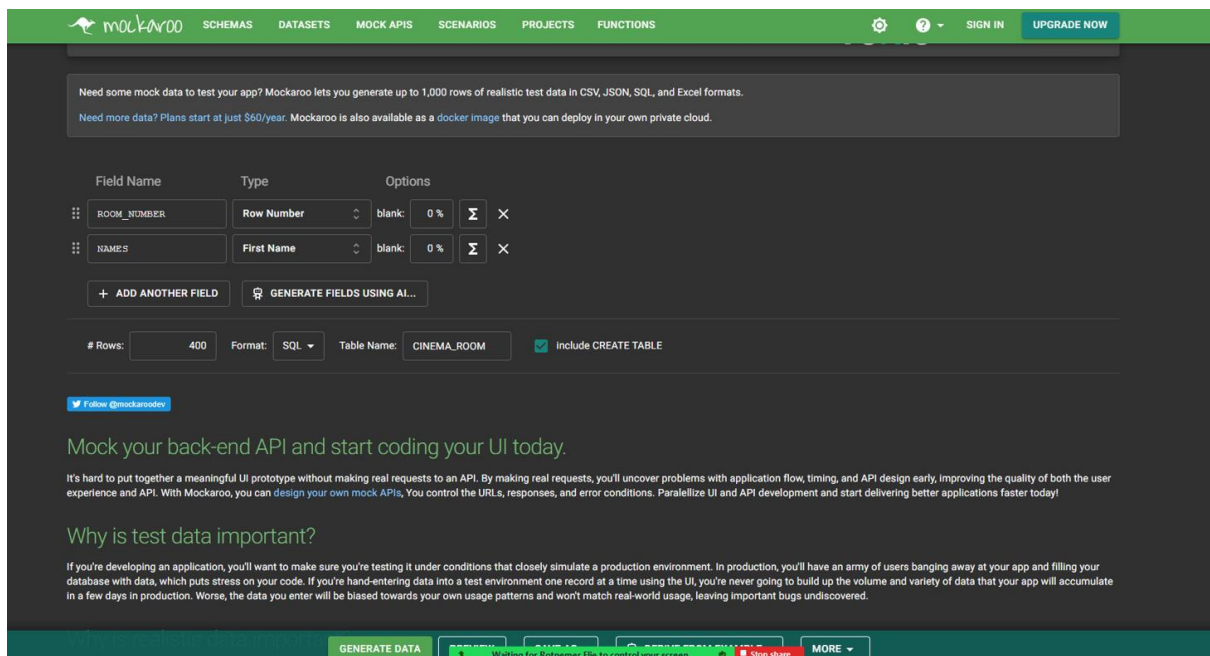
מגישים: ישראל שלמה 315130344
אלי רוטנמר 209572676
אלמוג שפיגל 315748871

מיני פרויקט בבסיסי נתונים

הכנסה פייתון:



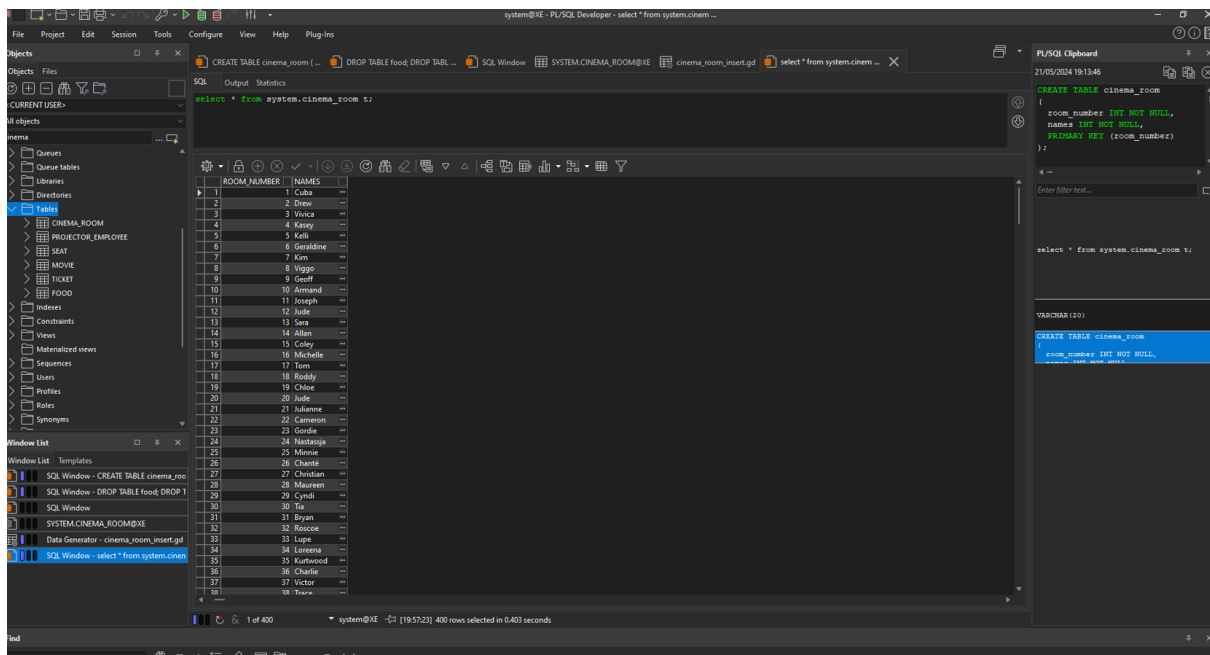
הכנסה mackroo הקובץ נמצא בגיט:



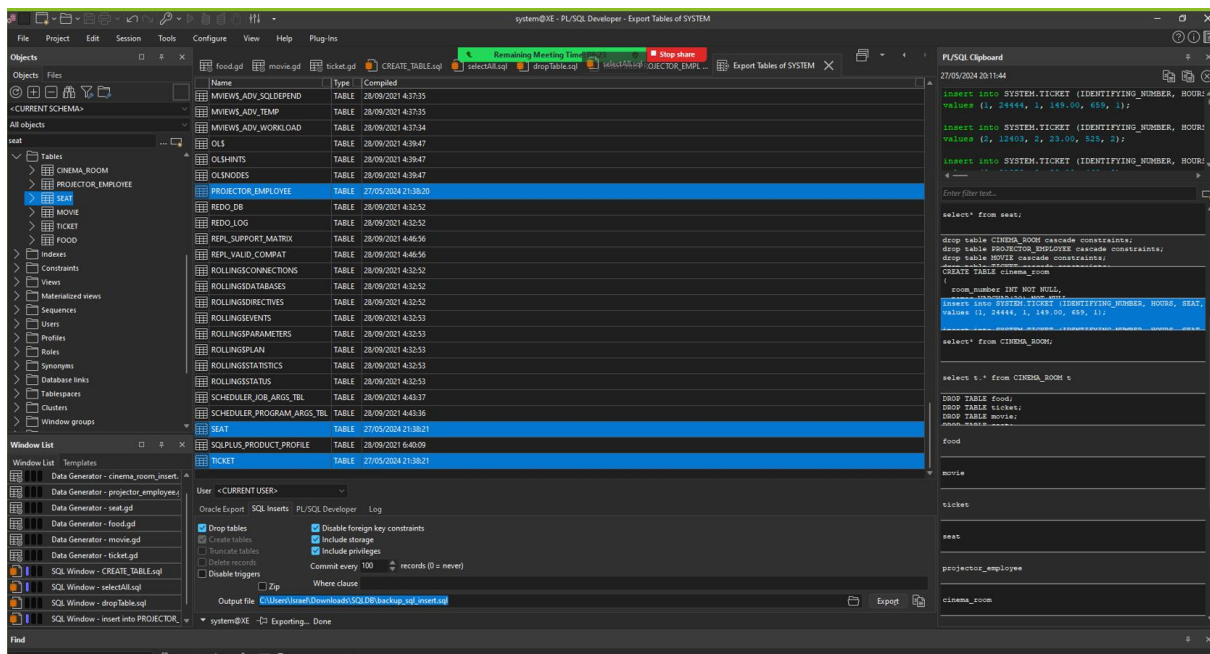
מגישים: ישראל שלמה 315130344
אלי רוטנמר 209572676
אלמוג שפיגל 315748871

מיני פרויקט בבסיסי נתונים

הכנסה ע"י data generator



גיבוי נתונים ושחזור נתונים:



קובץ השחזור נמצא בגיט

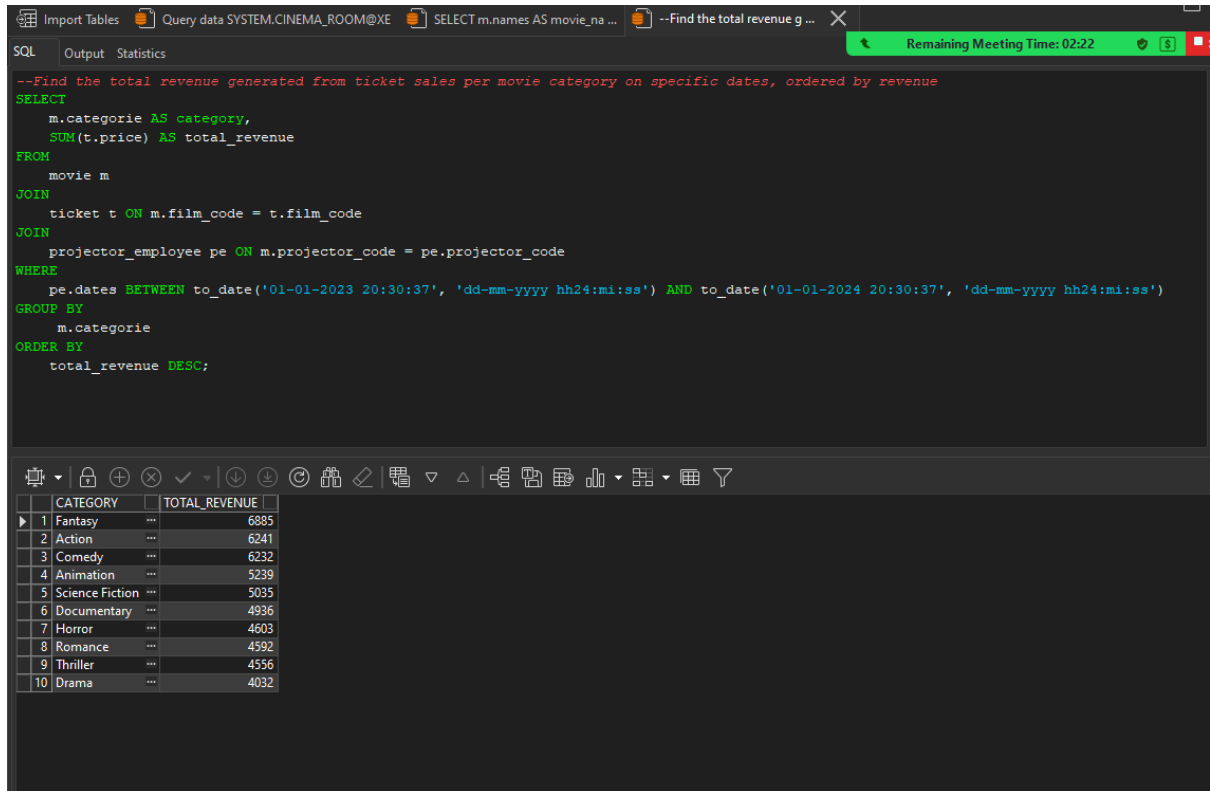
מגישים: ישראל שלמה 315130344
אלי רוטנמר 209572676
אלמוג שפיגל 315748871

מיני פרויקט בבסיסי נתונים

שלב ב:

4 שאילתות SELECT

חשב את שיעור התפוסה עבור כל חדר קולנוע וכן כרטיס שנמכר:



The screenshot shows a SQL IDE window with a query editor and a results pane. The query is as follows:

```
--Find the total revenue generated from ticket sales per movie category on specific dates, ordered by revenue
SELECT
  m.categorie AS category,
  SUM(t.price) AS total_revenue
FROM
  movie m
JOIN
  ticket t ON m.film_code = t.film_code
JOIN
  projector_employee pe ON m.projector_code = pe.projector_code
WHERE
  pe.dates BETWEEN to_date('01-01-2023 20:30:37', 'dd-mm-yyyy hh24:mi:ss') AND to_date('01-01-2024 20:30:37', 'dd-mm-yyyy hh24:mi:ss')
GROUP BY
  m.categorie
ORDER BY
  total_revenue DESC;
```

The results pane displays a table with the following data:

	CATEGORY	TOTAL_REVENUE
1	Fantasy	6885
2	Action	6241
3	Comedy	6232
4	Animation	5239
5	Science Fiction	5035
6	Documentary	4936
7	Horror	4603
8	Romance	4592
9	Thriller	4556
10	Drama	4032

מגשים: ישראל שלמה 315130344

אלי רוטנמר 209572676

אלמוג שפיגל 315748871

שאלתה שמחזירה את סך ההכנסות שנוצרו ממכירת כרטיסים לכל קטגוריית סרט בתאריכים, לפי הכנסה:

```
--Find the total revenue generated from ticket sales per movie category on specific dates, ordered by revenue
SELECT
    m.categorie AS category,
    SUM(t.price) AS total_revenue
FROM
    movie m
JOIN
    ticket t ON m.film_code = t.film_code
JOIN
    projector_employee pe ON m.projector_code = pe.projector_code
WHERE
    pe.dates BETWEEN to_date('01-01-2023 20:30:37', 'dd-mm-yyyy hh24:mi:ss') AND to_date('01-01-2024 20:30:37', 'dd-mm-yyyy hh24:mi:ss')
GROUP BY
    m.categorie
ORDER BY
    total_revenue DESC;
```

	CATEGORY	TOTAL_REVENUE
1	Fantasy	6885
2	Action	6241
3	Comedy	6232
4	Animation	5239
5	Science Fiction	5035
6	Documentary	4936
7	Horror	4603
8	Romance	4592
9	Thriller	4556
10	Drama	4032

שאלתה שמחזירה את מחיר לפי משך(אורך) הסרט:

```
--receipt price according to movie duration
SELECT
    m.duration AS movie_duration,
    SUM(f.price) AS total_receipt_price
FROM
    food f
JOIN
    ticket t ON f.identifying_number = t.identifying_number
JOIN
    movie m ON t.film_code = m.film_code
GROUP BY
    m.duration
ORDER BY
    m.duration;
```

	MOVIE_DURATION	TOTAL_RECEIPT_PRICE
1	60	93
2	61	80
3	62	254
4	63	206
5	64	29
6	65	168
7	66	122
8	67	104
9	68	11
10	69	61
11	70	316
12	71	244
13	72	108
14	73	151
15	74	50
16	75	187
17	76	188
18	77	54
19	78	81
20	79	124
21	80	91
22	81	64
23	82	9

system@XE [19:46:57] 23 rows selected in 0.080 seconds (more...)

שאלתה שמחזירה את מספר תחזיות שהתרחשו בין 2023 ל-2024 בכל חדר:

```
--number of projections that happened between 2023 and 2024 in each room
SELECT
  COUNT(pe.projector_code) AS number_of_projections,
  pe.room_numbeer AS room_number
FROM
  projector_employee pe
WHERE
  pe.dates BETWEEN to_date('01-01-2023 20:30:37', 'dd-mm-yyyy hh24:mi:ss') AND to_date('01-01-2024 20:30:37', 'dd-mm-yyyy hh24:mi:ss')
GROUP BY
  pe.room_numbeer
ORDER BY
  number_of_projections DESC;
```

	NUMBER_OF_PROJECTIONS	ROOM_NUMBER
1	28	15
2	27	3
3	27	8
4	25	13
5	24	18
6	22	5
7	22	11
8	22	17
9	21	2
10	20	1
11	19	4
12	19	19
13	18	10
14	18	7
15	17	9
16	16	6
17	14	14
18	14	20
19	14	16
20	12	12

Mute Start Video Participants Chat New share Pause Share

You are screen sharing

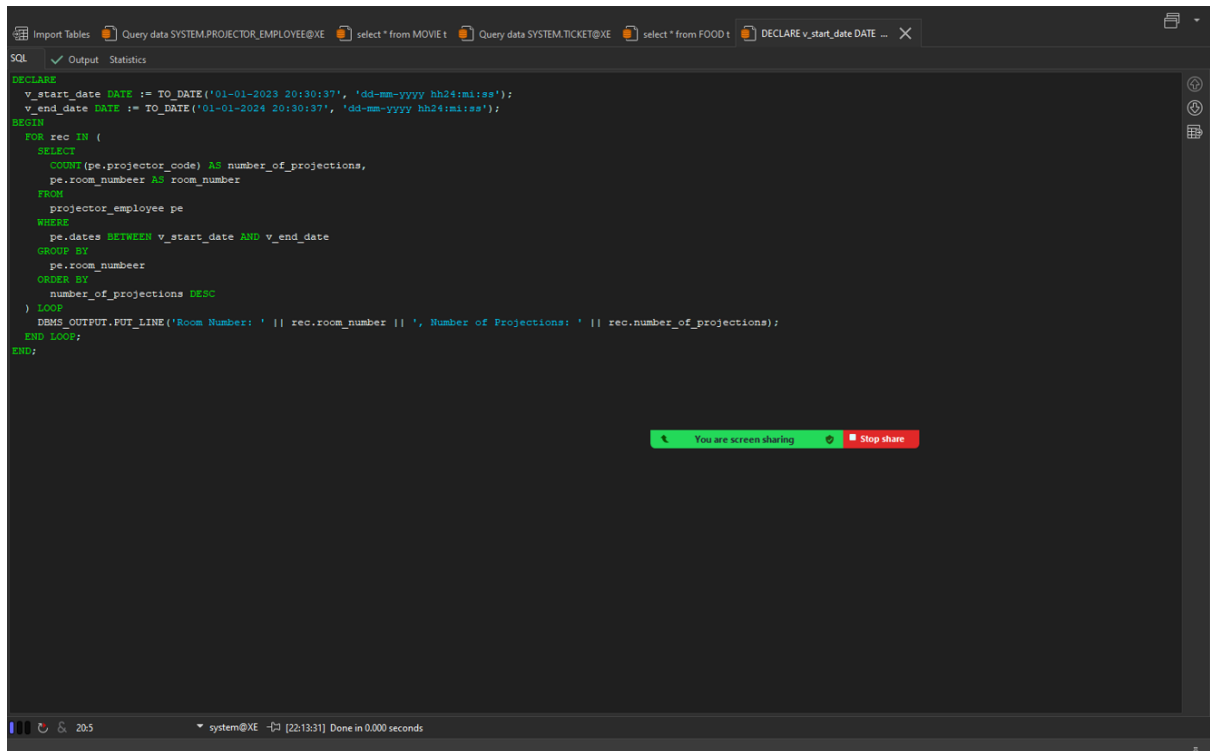
1:73 system@XE [19:57:20] 20 rows selected in 0.049 seconds

מגישים: ישראל שלמה 315130344
אלי רוטנמר 209572676
אלמוג שפיגל 315748871

מיני פרויקט בבסיסי נתונים

שאלות עם פרמטרים.

שאלתה 1:

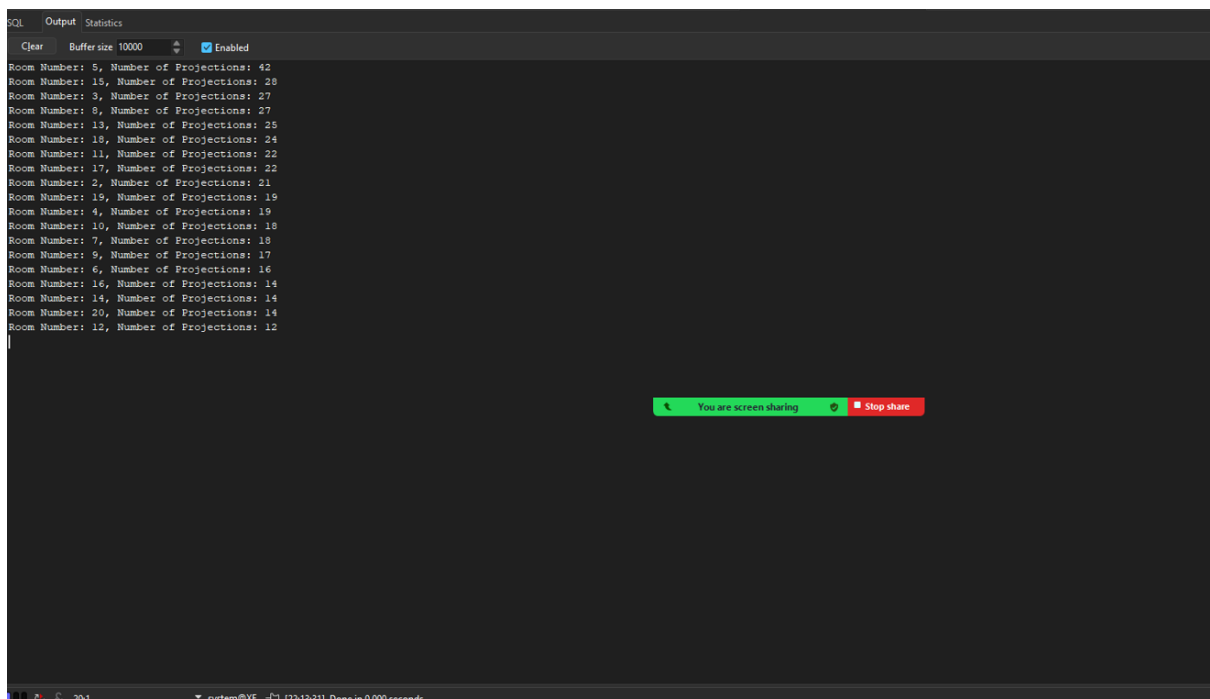


```
SQL | Import Tables | Query data SYSTEM.PROJECTOR_EMPLOYEE@XE | select * from MOVIE t | Query data SYSTEM.TICKET@XE | select * from FOOD t | DECLARE v_start_date DATE ... | X
```

```
SQL | Output | Statistics
```

```
DECLARE
v_start_date DATE := TO_DATE('01-01-2023 20:30:37', 'dd-mm-yyyy hh24:mi:ss');
v_end_date DATE := TO_DATE('01-01-2024 20:30:37', 'dd-mm-yyyy hh24:mi:ss');
BEGIN
FOR rec IN (
SELECT
COUNT(pe.projector_code) AS number_of_projections,
pe.room_number AS room_number
FROM
projector_employee pe
WHERE
pe.dates BETWEEN v_start_date AND v_end_date
GROUP BY
pe.room_number
ORDER BY
number_of_projections DESC
) LOOP
DBMS_OUTPUT.PUT_LINE('Room Number: ' || rec.room_number || ', Number of Projections: ' || rec.number_of_projections);
END LOOP;
END;
```

20:5 system@XE [22:13:31] Done in 0.000 seconds



```
SQL | Output | Statistics
```

Clear Buffer size 10000 Enabled

```
Room Number: 5, Number of Projections: 42
Room Number: 15, Number of Projections: 28
Room Number: 3, Number of Projections: 27
Room Number: 8, Number of Projections: 27
Room Number: 13, Number of Projections: 25
Room Number: 18, Number of Projections: 24
Room Number: 11, Number of Projections: 22
Room Number: 17, Number of Projections: 22
Room Number: 2, Number of Projections: 21
Room Number: 19, Number of Projections: 19
Room Number: 4, Number of Projections: 19
Room Number: 10, Number of Projections: 18
Room Number: 7, Number of Projections: 18
Room Number: 9, Number of Projections: 17
Room Number: 6, Number of Projections: 16
Room Number: 16, Number of Projections: 14
Room Number: 14, Number of Projections: 14
Room Number: 20, Number of Projections: 14
Room Number: 12, Number of Projections: 12
```

20:1 system@XE [22:13:31] Done in 0.000 seconds

מגישים: ישראל שלמה 315130344
אלי רוטנמר 209572676
אלמוג שפיגל 315748871

מיני פרויקט בבסיסי נתונים

שאלתה 2:

```
SQL | Output | Statistics
DECLARE
BEGIN
FOR rec IN (
SELECT
m.duration AS movie_duration,
SUM(f.price) AS total_receipt_price
FROM
food f
JOIN
ticket t ON f.identifying_number = t.identifying_number
JOIN
movie m ON t.film_code = m.film_code
GROUP BY
m.duration
ORDER BY
m.duration
) LOOP
DBMS_OUTPUT.PUT_LINE('Movie Duration: ' || rec.movie_duration || ', Total Receipt Price: ' || rec.total_receipt_price);
END LOOP;
END;
```

```
SQL | Output | Statistics
Clear | Buffer size 10000 | Enabled
Movie Duration: 60, Total Receipt Price: 57
Movie Duration: 61, Total Receipt Price: 80
Movie Duration: 62, Total Receipt Price: 52
Movie Duration: 63, Total Receipt Price: 84
Movie Duration: 64, Total Receipt Price: 29
Movie Duration: 65, Total Receipt Price: 33
Movie Duration: 66, Total Receipt Price: 47
Movie Duration: 67, Total Receipt Price: 74
Movie Duration: 68, Total Receipt Price: 11
Movie Duration: 69, Total Receipt Price: 61
Movie Duration: 70, Total Receipt Price: 137
Movie Duration: 71, Total Receipt Price: 86
Movie Duration: 73, Total Receipt Price: 151
Movie Duration: 74, Total Receipt Price: 5
Movie Duration: 75, Total Receipt Price: 77
Movie Duration: 76, Total Receipt Price: 133
Movie Duration: 78, Total Receipt Price: 7
Movie Duration: 79, Total Receipt Price: 124
Movie Duration: 80, Total Receipt Price: 91
Movie Duration: 81, Total Receipt Price: 64
Movie Duration: 82, Total Receipt Price: 9
Movie Duration: 83, Total Receipt Price: 69
Movie Duration: 84, Total Receipt Price: 144
Movie Duration: 85, Total Receipt Price: 33
Movie Duration: 86, Total Receipt Price: 14
Movie Duration: 87, Total Receipt Price: 86
Movie Duration: 88, Total Receipt Price: 73
Movie Duration: 89, Total Receipt Price: 73
Movie Duration: 90, Total Receipt Price: 20
Movie Duration: 91, Total Receipt Price: 174
Movie Duration: 92, Total Receipt Price: 62
Movie Duration: 93, Total Receipt Price: 48
Movie Duration: 94, Total Receipt Price: 86
Movie Duration: 95, Total Receipt Price: 99
Movie Duration: 96, Total Receipt Price: 60
Movie Duration: 97, Total Receipt Price: 44
Movie Duration: 98, Total Receipt Price: 113
Movie Duration: 100, Total Receipt Price: 13
Movie Duration: 101, Total Receipt Price: 38
Movie Duration: 103, Total Receipt Price: 7
Movie Duration: 104, Total Receipt Price: 177
Movie Duration: 105, Total Receipt Price: 168
Movie Duration: 106, Total Receipt Price: 146
system@XE [22:16:04] Done in 0.031 seconds
```

מגישים: ישראל שלמה 315130344
אלי רוטנמר 209572676
אלמוג שפיגל 315748871

מיני פרויקט בבסיסי נתונים

שאלה 3:

```
DECLARE
v_start_date DATE := TO_DATE('01-01-2023 20:30:37', 'dd-mm-yyyy hh24:mi:ss');
v_end_date DATE := TO_DATE('01-01-2024 20:30:37', 'dd-mm-yyyy hh24:mi:ss');
BEGIN
FOR rec IN (
SELECT
m.categorie AS category,
SUM(t.price) AS total_revenue
FROM
movie m
JOIN
ticket t ON m.film_code = t.film_code
JOIN
projector_employee pe ON m.film_code = pe.projector_code
WHERE
pe.dates BETWEEN v_start_date AND v_end_date
GROUP BY
m.categorie
ORDER BY
total_revenue DESC
) LOOP
DBMS_OUTPUT.PUT_LINE('Category: ' || rec.category || ', Total Revenue: ' || rec.total_revenue);
END LOOP;
END;
```

SQL	Output	Statistics
	Clear	Buffer size 10000 ☒ Enabled
	Category: Fantasy, Total Revenue: 6885	
	Category: Action, Total Revenue: 6241	
	Category: Comedy, Total Revenue: 6232	
	Category: Animation, Total Revenue: 5239	
	Category: Science Fiction, Total Revenue: 5035	
	Category: Documentary, Total Revenue: 4936	
	Category: Horror, Total Revenue: 4603	
	Category: Romance, Total Revenue: 4592	
	Category: Thriller, Total Revenue: 4556	
	Category: Drama, Total Revenue: 4032	

מגשים: ישראל שלמה 315130344
אלי רוטנמר 209572676
אלמוג שפיגל 315748871

2 שאלות המחיקה:

שאלתה 1 מוחקת סוג מסוים של מזון:

```
DELETE FROM food
WHERE food_items LIKE '%Salmon%';
```

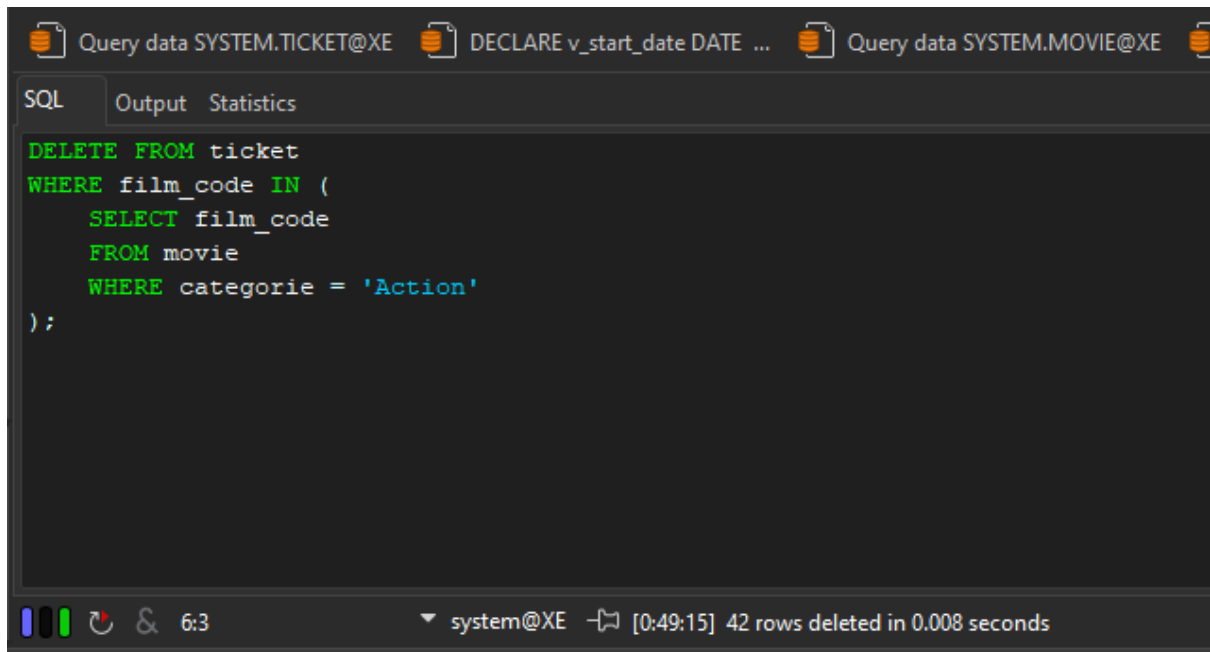
system@XE [21:18:52] 158 rows deleted in 0.039 seconds

,8,7

5

שאלתה 2:

מחק את כל הכרטיסים לקטגוריית סרטים ספציפית:



The screenshot shows the Oracle SQL Developer interface. At the top, there are three tabs: 'Query data SYSTEM.TICKET@XE', 'DECLARE v_start_date DATE ...', and 'Query data SYSTEM.MOVIE@XE'. The 'SQL' tab is active, displaying the following SQL query:

```
DELETE FROM ticket
WHERE film_code IN (
  SELECT film_code
  FROM movie
  WHERE categorie = 'Action'
);
```

Below the query, the 'Output' and 'Statistics' tabs are visible. The 'Statistics' tab shows the execution results:

system@XE [0:49:15] 42 rows deleted in 0.008 seconds

2 שאלות העדכון:

שאלתה של עדכון והחלפה סוג מסוים של מזון:

```
UPDATE food
SET food_items = REPLACE(food_items, 'Soda', 'Coke')
WHERE food_items LIKE '%Soda%';
```

שאלתה שמעדכנת את מספר חדרי הקולנוע:

```
UPDATE projector_employee
SET room_numbeer = 5
WHERE room_numbeer = 1;
```

Waiting for R

system@XE [21:32:12] 20 rows updated in 0.006 seconds

שלב ג'

הפונקציה הראשונה:

פונקציה שמחשבת את המספר הכולל של מושבים בחדר נתון:

```
1  --Function 1
2  --This function calculates the total number of seats in a given room.
3
4  CREATE OR REPLACE FUNCTION calculate_total_seats(room_id IN INTEGER) RETURN INTEGER IS
5  total_seats INTEGER := 0;
6  CURSOR seat_cursor IS
7  SELECT COUNT(*) AS seat_count
8  FROM SEAT
9  WHERE room_number = room_id;
10 BEGIN
11 OPEN seat_cursor;
12 FETCH seat_cursor INTO total_seats;
13 CLOSE seat_cursor;
14 RETURN total_seats;
15 EXCEPTION
16 WHEN NO_DATA_FOUND THEN
17 RETURN 0;
18 WHEN OTHERS THEN
19 RETURN -1;
20 END calculate_total_seats;
```

פורצדורה 1:

מעדכנת את המחיר של מוצרי מזון באחוז נתון

```
1  --Procedure 1
2  --This procedure updates the price of food items by a given percentage.
3
4  CREATE OR REPLACE PROCEDURE update_food_prices(percentage IN NUMBER) IS
5  BEGIN
6  UPDATE FOOD
7  SET price = price * (1 + percentage / 100);
8  COMMIT;
9  EXCEPTION
10 WHEN OTHERS THEN
11 ROLLBACK;
12 RAISE;
13 END update_food_prices;
```

מגשים: ישראל שלמה 315130344
אלי רוטנר 209572676
אלמוג שפיגל 315748871

מיני פרויקט בבסיסי נתונים

תוכנית ראשית 1:

```
--Main Program 1
--Calls the calculate_total_seats function and the update_food_prices procedure.

BEGIN
  DECLARE
    room_id INTEGER := 1;
    total_seats INTEGER;
  BEGIN
    total_seats := calculate_total_seats(room_id);
    DBMS_OUTPUT.PUT_LINE('Total seats in room ' || room_id || ': ' || total_seats);

    update_food_prices(10);
    DBMS_OUTPUT.PUT_LINE('Food prices updated by 10%.');
  EXCEPTION
    WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE('An error occurred in the main program.');
```

```
END;
END;
```

ממסד הנתונים לפני:

SQL Output Statistics

```
select * from FOOD t
```

	FOOD_ITEMS	PRICE	RECEIPT	IDENTIFYING_NUMBER
1	Salmon	21	1	1
2	Soda,Juice,Bagel	9	2	2
3	Juice,Soup,Tea,Steak,Salmon,Pizza,Soda,Bagel	81	3	3
4	Steak,Salmon	41	4	4
5	Muffin,Pizza,Juice,Burger,Steak	56	5	5
6	Muffin,Latte,Tea,Salad	26	6	6
7	Salmon,Soda,Muffin,Bagel	28	7	7
8	Soup	10	8	8
9	Salmon,Steak,Pizza	58	9	9
10	Latte,Pasta,Salmon,Smoothie,Muffin,Soup,Salad,Juice,Tea	83	10	10
11	Juice,Latte,Burger,Pizza,Pasta,Smoothie,Muffin,Soda	67	11	11
12	Muffin,Smoothie,Bagel,Pasta,Soda	31	12	12
13	Soda	1	13	13
14	Steak,Salad,Soup,Pasta,Latte,Muffin,Juice,Salmon,Soda,Smoothie	101	14	14
15	Latte,Salad,Soup,Soda	31	15	15
16	Smoothie,Latte	13	16	16
17	Tea	3	17	17
18	Latte,Bagel,Soda	12	18	18
19	Smoothie,Soup,Salad,Pasta,Juice,Latte,Tea,Steak	81	19	19
20	Latte,Pizza,Muffin,Juice,Steak,Soup	63	20	20
21	Pizza,Smoothie,Tea,Salmon,Soda	51	21	21
22	Tea,Salad,Bagel,Juice,Soup,Steak,Soda,Muffin,Latte,Burger	77	22	22
23	Bagel,Soup,Muffin,Juice,Smoothie	25	23	23
24	Soda,Muffin	4	24	24
25	Pizza,Muffin,Latte,Juice,Soda,Burger,Soup,Pasta,Steak	91	25	25
26	Soda,Tea,Juice,Pizza	28	26	26
27	Steak,Pasta,Smoothie,Burger,Salmon,Soda,Juice,Bagel,Soup	92	27	27
28	Burger,Salmon,Smoothie,Soup,Pasta,Juice,Muffin,Pizza,Salad	98	28	28
29	Soup,Steak,Soda,Salad,Salmon,Tea,Smoothie,Juice	80	29	29
30	Salmon,Soda	22	30	30
31	Soda,Bagel,Juice,Salmon,Latte,Smoothie	43	31	31
32	Salad,Soda,Soup,Juice,Pasta,Muffin,Latte	53	32	32
33	Salad,Tea,Muffin,Pasta,Burger,Juice,Smoothie,Bagel,Soda	62	33	33
34	Latte,Soup,Salmon,Salad,Burger,Pasta,Steak,Soda	98	34	34

1:1 system@XE [21:42:44] 34 rows selected in 0.139 seconds (more...)

מגישים: ישראל שלמה 315130344
אלי רוטנמר 209572676
אלמוג שפיגל 315748871

מיני פרויקט בבסיסי נתונים

ממסד הנתונים אחריו:

```
select * from FOOD t
```

	FOOD_ITEMS	PRICE	RECEIPT	IDENTIFYING_NUMBER
1	Salmon	23	1	1
2	Soda, Juice, Bagel	10	2	2
3	Juice, Soup, Tea, Steak, Salmon, Pizza, Soda, Bagel	89	3	3
4	Steak, Salmon	45	4	4
5	Muffin, Pizza, Juice, Burger, Steak	62	5	5
6	Muffin, Latte, Tea, Salad	29	6	6
7	Salmon, Soda, Muffin, Bagel	31	7	7
8	Soup	11	8	8
9	Salmon, Steak, Pizza	64	9	9
10	Latte, Pasta, Salmon, Smoothie, Muffin, Soup, Salad, Juice, Tea	91	10	10
11	Juice, Latte, Burger, Pizza, Pasta, Smoothie, Muffin, Soda	74	11	11
12	Muffin, Smoothie, Bagel, Pasta, Soda	34	12	12
13	Soda	1	13	13
14	Steak, Salad, Soup, Pasta, Latte, Muffin, Juice, Salmon, Soda, Smoothie	111	14	14
15	Latte, Salad, Soup, Soda	34	15	15
16	Smoothie, Latte	14	16	16
17	Tea	3	17	17
18	Latte, Bagel, Soda	13	18	18
19	Smoothie, Soup, Salad, Pasta, Juice, Latte, Tea, Steak	89	19	19
20	Latte, Pizza, Muffin, Juice, Steak, Soup	69	20	20
21	Pizza, Smoothie, Tea, Salmon, Soda	56	21	21
22	Tea, Salad, Bagel, Juice, Soup, Steak, Soda, Muffin, Latte, Burger	85	22	22
23	Bagel, Soup, Muffin, Juice, Smoothie	28	23	23
24	Soda, Muffin	4	24	24
25	Pizza, Muffin, Latte, Juice, Soda, Burger, Soup, Pasta, Steak	100	25	25
26	Soda, Tea, Juice, Pizza	31	26	26
27	Steak, Pasta, Smoothie, Burger, Salmon, Soda, Juice, Bagel, Soup	101	27	27
28	Burger, Salmon, Smoothie, Soup, Pasta, Juice, Muffin, Pizza, Salad	108	28	28
29	Soup, Steak, Soda, Salad, Salmon, Tea, Smoothie, Juice	88	29	29
30	Salmon, Soda	24	30	30
31	Soda, Bagel, Juice, Salmon, Latte, Smoothie	47	31	31
32	Salad, Soda, Soup, Juice, Pasta, Muffin, Latte	58	32	32
33	Salad, Tea, Muffin, Pasta, Burger, Juice, Smoothie, Bagel, Soda	68	33	33
34	Latte, Soup, Salmon, Salad, Burger, Pasta, Steak, Soda	108	34	34

1 of 34 system@XE [21:43:45] 34 rows selected in 0.343 seconds (more...)

מגשים: ישראל שלמה 315130344
אלי רוטנמר 209572676
אלמוג שפיגל 315748871

מיני פרויקט בבסיסי נתונים

פונקציה 2:

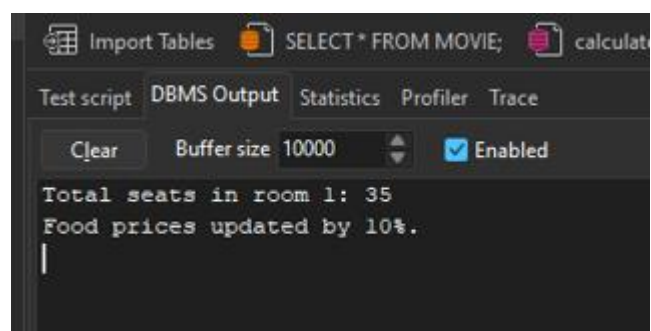
הפונקציה מחשבת את סך ההכנסות ממכירת כרטיסים לחדר ספציפי. ומשתמש "בסמן מפורש" ומטפל בחריגים.

```
1 --Function 2
2 --This function calculates the total revenue from ticket sales for a specific room. It uses an explicit cursor and handles exceptions.
3
4 CREATE OR REPLACE FUNCTION calculate_room_revenue(room_id IN INTEGER) RETURN NUMBER IS
5     total_revenue NUMBER := 0;
6     CURSOR ticket_cursor IS
7         SELECT t.price
8         FROM TICKET t
9         JOIN MOVIE m ON t.film_code = m.film_code
10        WHERE m.room_number = room_id;
11     ticket_price TICKET.price%TYPE;
12 BEGIN
13     OPEN ticket_cursor;
14     LOOP
15         FETCH ticket_cursor INTO ticket_price;
16         EXIT WHEN ticket_cursor%NOTFOUND;
17         total_revenue := total_revenue + ticket_price;
18     END LOOP;
19     CLOSE ticket_cursor;
20     RETURN total_revenue;
21 EXCEPTION
22     WHEN NO_DATA_FOUND THEN
23         RETURN 0;
24     WHEN OTHERS THEN
25         RETURN -1;
26 END calculate_room_revenue;
```

פרוצדורה 2:

מעדכן את משך הסרט ומתעד. הוא משתמש בפקודות DML, לולאות וטיפול בחריגים.

```
1 --This procedure updates the duration of a movie and logs a message. It uses DML commands, loops, and exception handling.
2
3 CREATE OR REPLACE PROCEDURE update_movie_duration(
4     p_film_code IN INTEGER,
5     p_new_duration IN INTEGER
6 ) IS
7     old_duration MOVIE.duration%TYPE;
8 BEGIN
9     SELECT duration INTO old_duration FROM MOVIE WHERE film_code = p_film_code;
10
11     UPDATE MOVIE
12     SET duration = p_new_duration
13     WHERE film_code = p_film_code;
14
15     DBMS_OUTPUT.PUT_LINE('Updated movie duration from ' || old_duration || ' to ' || p_new_duration);
16     COMMIT;
17 EXCEPTION
18     WHEN NO_DATA_FOUND THEN
19         DBMS_OUTPUT.PUT_LINE('Movie with this code does not exist.');
```



תוכנית ראשית 2:

```
1
2 --This main program calls the calculate_room_revenue function and the update_movie_duration procedure. It handles exceptions and outputs results.
3
4 BEGIN
5   DECLARE
6     room_id INTEGER := 1;
7     film_code INTEGER := 100;
8     new_duration INTEGER := 150;
9     total_revenue NUMBER;
10
11 BEGIN
12   total_revenue := calculate_room_revenue(room_id);
13   DBMS_OUTPUT.PUT_LINE('Total revenue for room ' || room_id || ': ' || total_revenue);
14
15   update_movie_duration(film_code, new_duration);
16   DBMS_OUTPUT.PUT_LINE('Updated duration for film code ' || film_code || ' to ' || new_duration);
17 EXCEPTION
18   WHEN OTHERS THEN
19     DBMS_OUTPUT.PUT_LINE('An error occurred in the main program.');
```

מסדר הנתונים לפני:

```
select * from MOVIE t
```

	FILM_CODE	NAMES	DURATION	CATEGORIE	ROOM_NUMBER	PROJECTOR_CODE
1	1	Pierce	90	Fantasy	11	1
2	2	Praga	110	Fantasy	13	2
3	3	Peabo	138	Documentary	1	3
4	4	Leslie	146	Animation	6	4
5	5	Praga	117	Horror	15	5
6	6	Tal	155	Science Fiction	14	6
7	7	Madeline	95	Thriller	4	7
8	8	Tara	176	Action	20	8
9	9	Jaime	130	Fantasy	4	9
10	10	Tim	116	Horror	15	10
11	11	Howie	61	Horror	18	11
12	12	Joan	61	Horror	1	12
13	13	Sylvester	152	Comedy	4	13
14	14	Rosario	108	Science Fiction	1	14
15	15	Fairuza	177	Documentary	2	15
16	16	Rawlins	88	Horror	3	16
17	17	Morgan	79	Romance	3	17
18	18	Rene	136	Comedy	16	18
19	19	Gran	155	Action	18	19
20	20	Diamond	66	Animation	9	20
21	21	Daniel	160	Fantasy	8	21
22	22	Isaiah	84	Animation	14	22
23	23	Parker	112	Drama	15	23
24	24	Demi	168	Animation	11	24
25	25	Brent	140	Action	2	25
26	26	Suzi	75	Animation	5	26
27	27	Rodney	86	Horror	19	27
28	28	Eugene	70	Romance	2	28
29	29	George	101	Thriller	6	29
30	30	Dar	96	Documentary	5	30
31	31	Jet	143	Fantasy	2	31
32	32	Gwyneth	159	Documentary	9	32
33	33	Jennifer	94	Romance	14	33
34	34	Emma	115	Animation	5	34

1 of 401 system@XE names, varchar2(20), mandatory

מגישים: ישראל שלמה 315130344
אלי רוטנמר 209572676
אלמוג שפיגל 315748871

מיני פרויקט בבסיסי נתונים

SQL Output Statistics

```
select * from MOVIE t
```

	FILM_CODE	NAMES	DURATION	CATEGORIE	ROOM_NUMBER	PROJECTOR_CODE
1	1	Pierce	150	Fantasy	11	1
2	2	Praga	110	Fantasy	13	2
3	3	Peabo	138	Documentary	1	3
4	4	Leslie	146	Animation	6	4
5	5	Praga	117	Horror	15	5
6	6	Tal	155	Science Fiction	14	6
7	7	Madeline	95	Thriller	4	7
8	8	Tara	176	Action	20	8
9	9	Jaime	130	Fantasy	4	9
10	10	Tim	116	Horror	15	10
11	11	Howie	61	Horror	18	11
12	12	Joan	61	Horror	1	12
13	13	Sylvester	152	Comedy	4	13
14	14	Rosario	108	Science Fiction	1	14
15	15	Fairuza	177	Documentary	2	15
16	16	Rawlins	88	Horror	3	16
17	17	Morgan	79	Romance	3	17
18	18	Rene	136	Comedy	16	18
19	19	Gran	155	Action	18	19
20	20	Diamond	66	Animation	9	20
21	21	Daniel	160	Fantasy	8	21
22	22	Isaiah	84	Animation	14	22
23	23	Parker	112	Drama	15	23
24	24	Demi	168	Animation	11	24
25	25	Brent	140	Action	2	25
26	26	Suzi	75	Animation	5	26
27	27	Rodney	86	Horror	19	27
28	28	Eugene	70	Romance	2	28
29	29	George	101	Thriller	6	29
30	30	Dar	96	Documentary	5	30
31	31	Jet	143	Fantasy	2	31
32	32	Gwyneth	159	Documentary	9	32
33	33	Jennifer	94	Romance	14	33
34	34	Emma	115	Animation	5	34

1 of 34 system@XE [22:15:53] 34 rows selected in 0.090 seconds (more...)

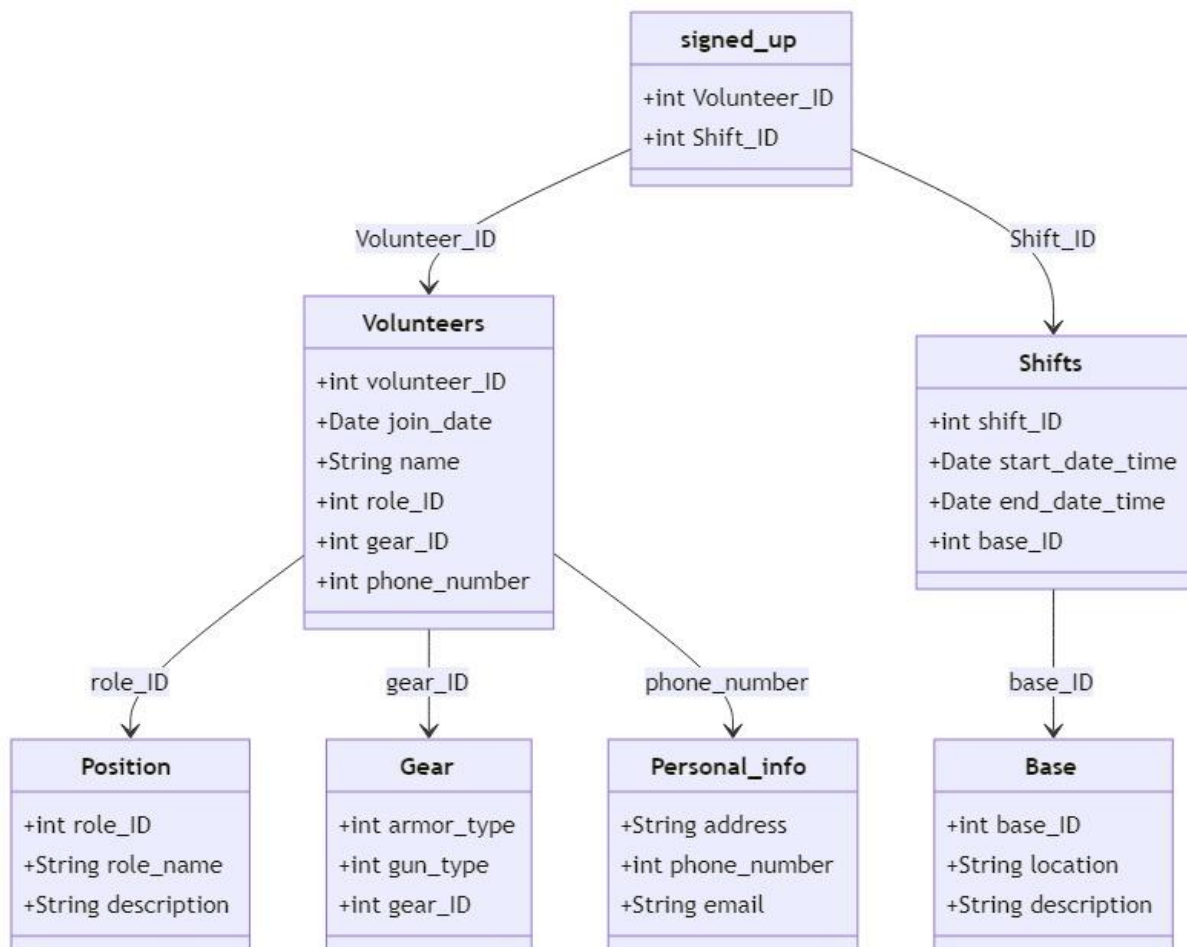
Test script DBMS Output Statistics Profiler Trace

Clear Buffer size 10000 ☒ Enabled

Total revenue for room 1: 3777
Updated movie duration from 90 to 150
|

שלב 4:

1. בעזרת הגיבוי בנינו את תרשים ה DSD הבא של המערך המתנדבים למשמר הלאומי:



Create table (של חזוג שני):

CREATE TABLE Position

(

role_ID INT NOT NULL,

role_name VARCHAR(100) NOT NULL,

description VARCHAR(1000) NOT NULL,

PRIMARY KEY (role_ID)

);

CREATE TABLE Base

(

מגשים: ישראל שלמה 315130344

אלי רוטנמר 209572676

אלמוג שפיגל 315748871

base_ID INT NOT NULL,

location VARCHAR(1000) NOT NULL,

description VARCHAR(1000) NOT NULL,

PRIMARY KEY (base_ID)

);

CREATE TABLE Gear

(

armor_type INT NOT NULL,

gun_type INT NOT NULL,

gear_ID INT NOT NULL,

PRIMARY KEY (gear_ID)

);

CREATE TABLE Personal_info

(

address VARCHAR(1000) NOT NULL,

phone_number INT NOT NULL,

email VARCHAR(500) NOT NULL,

PRIMARY KEY (phone_number)

);

CREATE TABLE Volunteers

(

volunteer_ID INT NOT NULL,

join_date DATE NOT NULL,

name VARCHAR(100) NOT NULL,

role_ID INT NOT NULL,

gear_ID INT NOT NULL,

phone_number INT NOT NULL,

PRIMARY KEY (volunteer_ID),

```
FOREIGN KEY (role_ID) REFERENCES Roles(role_ID),  
  
FOREIGN KEY (gear_ID) REFERENCES Gear(gear_ID),  
  
FOREIGN KEY (phone_number) REFERENCES Personal_info(phone_number)  
  
);
```

```
CREATE TABLE Shifts
```

```
(  
  
    shift_ID INT NOT NULL,  
  
    start_date_time DATE NOT NULL,  
  
    end_date_time DATE NOT NULL,  
  
    base_ID INT NOT NULL,  
  
    PRIMARY KEY (shift_ID),  
  
    FOREIGN KEY (base_ID) REFERENCES Base(base_ID)  
  
);
```

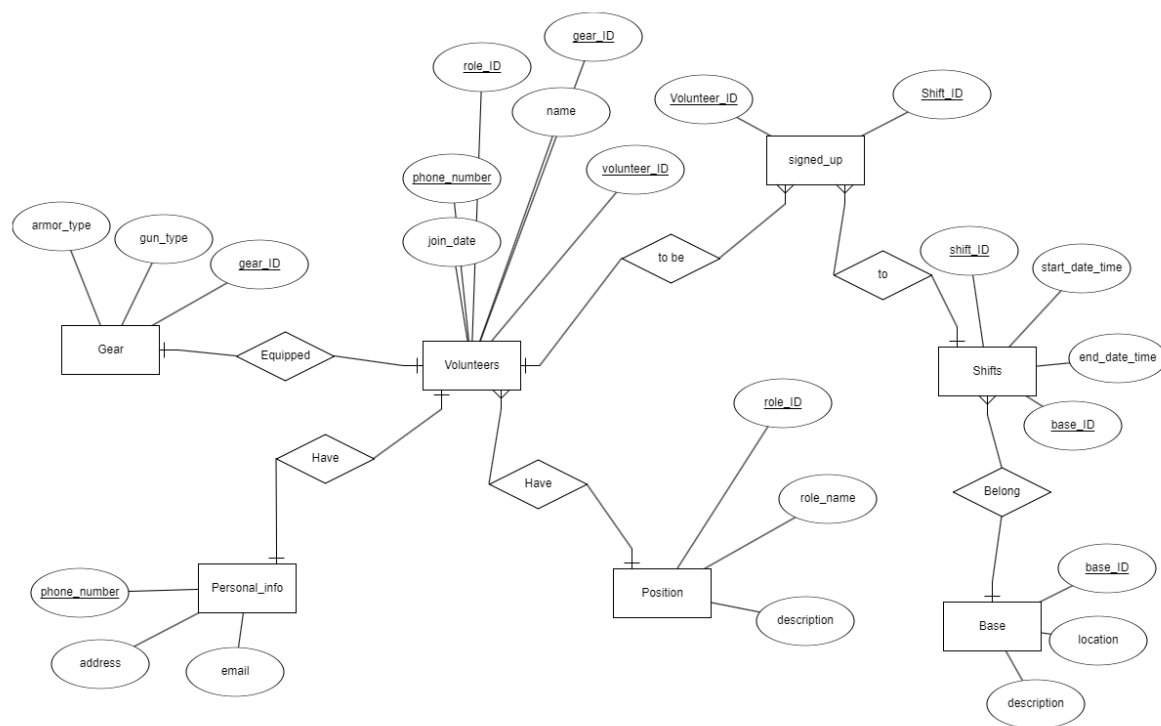
```
CREATE TABLE signed_up
```

```
(  
  
    Volunteer_ID INT NOT NULL,  
  
    Shift_ID INT NOT NULL,  
  
    PRIMARY KEY (Volunteer_ID, Shift_ID),  
  
    FOREIGN KEY (Volunteer_ID) REFERENCES Volunteers(volunteer_ID),  
  
    FOREIGN KEY (Shift_ID) REFERENCES Shifts(shift_ID)  
  
);
```

מגישים: ישראל שלמה 315130344
אלי רוטנמר 209572676
אלמוג שפיגל 315748871

מיני פרויקט בבסיסי נתונים

2. תרשים ERD



מיני פרויקט בבסיסי נתונים

The diagram is a complex Entity-Relationship (ER) model for a hospital database. It consists of several entities and their relationships:

- Entities and Attributes:**
 - Patient:** Attributes include name, date, time, and location. It has a primary key on name.
 - Nurse:** Attributes include name, date, time, and location. It has a primary key on name.
 - Doctor:** Attributes include name, date, time, and location. It has a primary key on name.
 - Shift:** Attributes include date, time, and location. It has a primary key on date and time.
 - Room:** Attributes include name, date, time, and location. It has a primary key on name.
 - Case:** Attributes include name, date, time, and location. It has a primary key on name.
- Relationships:**
 - has:** A relationship between Patient and Nurse, with a cardinality of 1:1.
 - works on:** A relationship between Nurse and Doctor, with a cardinality of 1:1.
 - is in:** A relationship between Doctor and Room, with a cardinality of 1:1.
 - has a:** A relationship between Doctor and Case, with a cardinality of 1:1.
 - works on:** A relationship between Nurse and Shift, with a cardinality of 1:1.
 - has a:** A relationship between Shift and Room, with a cardinality of 1:1.
- Other Features:**
 - The diagram includes various attributes like name, date, time, and location, which are used to identify entities and relationships.
 - Primary keys are indicated by a thick line connecting the attribute to the entity.
 - Foreign keys are indicated by a thin line connecting the attribute to the relationship.
 - Cardinalities are shown at the ends of the relationship lines.

ALTER TABLE CINEMA_ROOM

```
FOREIGN KEY (base_id)
```

```
;REFERENCES BASE (base_id)
```

The diagram illustrates a movie theater database schema and a user interface flow. The schema includes tables for tickets, films, receipts, projector employees, seats, cinema rooms, signed-up users, volunteers, shifts, positions, gear, personal info, and bases. The UI flow shows a user waiting for a movie, with buttons for 'Waiting for Rotmerer Eile to control your screen' and 'Stop share', leading to a 'cinema room' table.

Database Schema:

- ticket**
 - identifying number (PK)
 - hour
 - seat
 - price
 - movie name
 - film code (FK)
- film**
 - film code (PK)
 - movie name
 - duration
 - category
- receipt**
 - food
 - drinks
 - identifying number (FK)
- projector_employee**
 - projector code (FK)
 - date
 - time
 - room number
 - film code
- seat**
 - seat number (PK)
 - row number (U)
 - room number (FK)
- cinema_room**
 - room number (PK)
 - name
 - int base_ID
- signed_up**
 - +int Volunteer_ID
 - +int Shift_ID
- Volunteers**
 - +int volunteer_ID
 - +Date join_date
 - +String name
 - +int role_ID
 - +int gear_ID
 - +int phone_number
- Shifts**
 - +int shift_ID
 - +Date start_date_time
 - +Date end_date_time
 - +int base_ID
- Position**
 - +int role_ID
 - +String role_name
 - +String description
- Gear**
 - +int armor_type
 - +int gun_type
 - +int gear_ID
- Personal_info**
 - +String address
 - +int phone_number
 - +String email
- Base**
 - +int base_ID
 - +String location
 - +String description

User Interface Flow:

- ticket** (identifying number, hour, seat, price, movie name, film code) leads to **film** (film code, movie name, duration, category).
- film** leads to **cinema_room** (room number, name, int base_ID).
- cinema_room** leads to **signed_up** (int Volunteer_ID, int Shift_ID).
- signed_up** leads to **Volunteers** (int volunteer_ID, Date join_date, String name, int role_ID, int gear_ID, int phone_number) and **Shifts** (int shift_ID, Date start_date_time, Date end_date_time, int base_ID).
- Volunteers** leads to **Position** (int role_ID, String role_name, String description), **Gear** (int armor_type, int gun_type, int gear_ID), **Personal_info** (String address, int phone_number, String email), and **Base** (int base_ID, String location, String description).
- Shifts** leads to **Base**.
- Base** leads to **Base** (int base_ID, String location, String description).

User Interface Flow (Continued):

- Waiting for Rotmerer Eile to control your screen** (button) leads to **cinema_room**.
- Stop share** (button) leads to **cinema_room**.
- cinema_room** leads to **seat** (seat number, row number, room number).
- seat** leads to **projector_employee** (projector code, date, time, room number, film code).
- projector_employee** leads to **receipt** (food, drinks, identifying number).
- receipt** leads to **ticket** (identifying number, hour, seat, price, movie name, film code).

מגשים: ישראל שלמה 315130344
אלי רוטנמר 209572676
אלמוג שפיגל 315748871

מיני פרויקט בבסיסי נתונים

הפונקציה שמכניסה לטבלה שלנו את ה: base_id

```
CREATE OR REPLACE FUNCTION get_random_base_id RETURN INTEGER IS
```

```
random_base_id INTEGER;
```

```
BEGIN
```

```
SELECT base_id
```

```
INTO random_base_id
```

```
FROM (
```

```
SELECT base_id
```

```
FROM BASE
```

```
ORDER BY DBMS_RANDOM.VALUE
```

```
)
```

```
WHERE ROWNUM = 1;
```

```
RETURN random_base_id;
```

```
END;
```

```
BEGIN
```

```
FOR rec IN (SELECT room_number FROM CINEMA_ROOM) LOOP
```

```
UPDATE CINEMA_ROOM
```

```
SET base_id = get_random_base_id()
```

```
WHERE room_number = rec.room_number;
```

```
END LOOP;
```

```
COMMIT;
```

```
END;
```

5. מבט ראשון מנקודת מבט של הזוג השני:

```
CREATE OR REPLACE VIEW VOLUNTEER_SHIFT_DETAILS AS
SELECT
    v.volunteer_id,
    v.name AS volunteer_name,
    v.phone_number,
    s.shift_id,
    s.start_date_time,
    s.end_date_time,
    b.base_id,
    b.location AS base_location,
    b.description AS base_description
FROM
    VOLUNTEERS v
JOIN
    SIGNED_UP su ON v.volunteer_id = su.volunteer_id
JOIN
    SHIFTS s ON su.shift_id = s.shift_id
JOIN
    BASE b ON s.base_id = b.base_id;
```

מגישים: ישראל שלמה 315130344
אלי רוטנמר 209572676
אלמוג שפיגל 315748871

מיני פרויקט בבסיסי נתונים

SQL Output Statistics

select * from VOLUNTEER_SHIFT_DETAILS

	VOLUNTEER_ID	VOLUNTEER_NAME	PHONE_NUMBER	SHIFT_ID	START_DATE_TIME	END_DATE_TIME	BASE_ID	BASE_LOCATION	BASE_DESCRIPTION
1	300650178	LeslieShandling	534284901	307541551	08/10/2023	05/08/2023	764079527	Buffalo	Antec Plus 660 ATX Mid tower 330w Power Supply
2	376634667	AlecKershaw	531124169	615866333	30/05/2023	03/06/2023	639946543	Conshohocken	Combo AMD XP 2700+ CPU ASUS A7N8X Deluxe NVIDIA Force2
3	308828796	WarrenHauser	538627138	572034903	11/03/2023	30/04/2023	639946543	Conshohocken	Combo AMD XP 2700+ CPU ASUS A7N8X Deluxe NVIDIA Force2
4	469226077	CaseyBacharach	536982267	764506310	30/04/2023	23/02/2023	639946543	Conshohocken	Combo AMD XP 2700+ CPU ASUS A7N8X Deluxe NVIDIA Force2
5	977094617	WilliamCobbs	533347394	764506310	30/04/2023	23/02/2023	639946543	Conshohocken	Combo AMD XP 2700+ CPU ASUS A7N8X Deluxe NVIDIA Force2
6	638266872	BarbaraParish	537159592	285486340	10/02/2023	13/04/2023	115736975	Gersthofen	Linksys Instant Wireless Network Adapter - Version 3.0 for Notebooks. (Me
7	578501243	CaryTettos	538830110	344639962	31/05/2023	06/11/2023	895591429	Dinslaken	Dynatron MicroFin DC1206BM-I AMD Athlon/Duron Fan w/ Copper Base
8	655356874	OzzyDay-Lewis	538263593	826491560	11/06/2023	01/05/2023	883554799	Erpe-Mere	Biostar U8688D P4 uATX M/B (VIA P4M266 478pin) with AMR;Support DI
9	57140582	BernardCattrell	539898546	826491560	11/06/2023	01/05/2023	883554799	Erpe-Mere	Biostar U8688D P4 uATX M/B (VIA P4M266 478pin) with AMR;Support DI
10	125203316	KrisFlanery	535394978	686672809	28/03/2023	27/10/2023	883554799	Erpe-Mere	Biostar U8688D P4 uATX M/B (VIA P4M266 478pin) with AMR;Support DI
11	257252001	JakeAbraham	537037527	826491560	11/06/2023	01/05/2023	883554799	Erpe-Mere	Biostar U8688D P4 uATX M/B (VIA P4M266 478pin) with AMR;Support DI
12	639672862	SalmaLindley	536042736	686672809	28/03/2023	27/10/2023	883554799	Erpe-Mere	Biostar U8688D P4 uATX M/B (VIA P4M266 478pin) with AMR;Support DI
13	580436161	BruceDel Toro	533283024	686672809	28/03/2023	27/10/2023	883554799	Erpe-Mere	Biostar U8688D P4 uATX M/B (VIA P4M266 478pin) with AMR;Support DI
14	950413481	VerticalFox	538919287	738867544	19/12/2023	15/06/2023	221935402	Irvine	AMDXP2200/128M/30G Black Complete System.
15	189375133	NicoleSevenfold	531281568	738867544	19/12/2023	15/06/2023	221935402	Irvine	AMDXP2200/128M/30G Black Complete System.
16	325055451	ChaleeKistofferson	534993864	628685669	11/08/2023	01/09/2023	467653037	Storington	PCI 56K V.92 Fax Modem
17	67721837	LorraineUggams	536210601	589111609	19/04/2023	16/03/2023	862072713	Frederiksberg	Combo Intel P4 1.8Ghz retail CPU + Gigabyte GA-8IE533 M/B + Fan.
18	314994995	RodMcCann	530442051	260692055	02/07/2023	08/07/2023	862072713	Frederiksberg	Combo Intel P4 1.8Ghz retail CPU + Gigabyte GA-8IE533 M/B + Fan.
19	238381426	JasonFranks	531818896	260692055	02/07/2023	08/07/2023	862072713	Frederiksberg	Combo Intel P4 1.8Ghz retail CPU + Gigabyte GA-8IE533 M/B + Fan.
20	951544298	MelanieFiorentino	537828436	542520861	16/10/2023	06/11/2023	301914323	Rueil-Malmaison	Combo Intel Celeron 1.3ghz Biostar M6VLQ m/b + fan.
21	125206388	JonathaCotton	530109419	923486122	06/10/2023	17/12/2023	507134835	Gattico	Intel P4 1.8Ghz (478) pin 512K 3 Year MFG warr.
22	861529245	GusCrimson	533403509	923486122	06/10/2023	17/12/2023	507134835	Gattico	Intel P4 1.8Ghz (478) pin 512K 3 Year MFG warr.
23	435364818	SonolAndau	532635955	923486122	06/10/2023	17/12/2023	507134835	Gattico	Intel P4 1.8Ghz (478) pin 512K 3 Year MFG warr.
24	451737131	WallaceYankovic	539856383	249424386	21/11/2023	13/02/2023	176100852	Lisbon	Gigabyte GA 70PXDW-c Dual AMD server board
25	525133659	TriniHopper	538851007	249424386	21/11/2023	13/02/2023	176100852	Lisbon	Gigabyte GA 70PXDW-c Dual AMD server board
26	331347938	LindseyHannah	534740622	610824383	23/11/2023	03/08/2023	240115315	Montreal	Combo AMD XP 1900+ Gigabyte GA-7VKML M/B + Fan
27	488488861	HexPreston	530421723	610824383	23/11/2023	03/08/2023	240115315	Montreal	Combo AMD XP 1900+ Gigabyte GA-7VKML M/B + Fan
28	789723635	RolandoRobbins	536210601	517067823	26/04/2023	09/10/2023	240115315	Montreal	Combo AMD XP 1900+ Gigabyte GA-7VKML M/B + Fan
29	507789069	RitaHaslam	531320992	142315637	10/12/2023	11/03/2023	360881982	Braintree	AMD XP 2400+ Retail box.
30	450634687	Jean-LucFerrer	531554530	207034524	02/09/2023	04/02/2023	360881982	Braintree	AMD XP 2400+ Retail box.
31	762442704	AidanLinney	536525677	142315637	10/12/2023	11/03/2023	360881982	Braintree	AMD XP 2400+ Retail box.
32	253786755	DelroyReynolds	535144823	207034524	02/09/2023	04/02/2023	360881982	Braintree	AMD XP 2400+ Retail box.
33	360593887	GranShue	536210601	124611647	23/05/2023	23/10/2023	247984877	Double Oak	2 Year Warranty On Combo's.
34	775209739	treyMyers	536686821	124611647	23/05/2023	23/10/2023	247984877	Double Oak	2 Year Warranty On Combo's.

system@XE [23:37:13] 34 rows selected in 0.207 seconds (more...)

שאלתא 1:

```
--List Volunteers and Their previous Shifts with Base Information

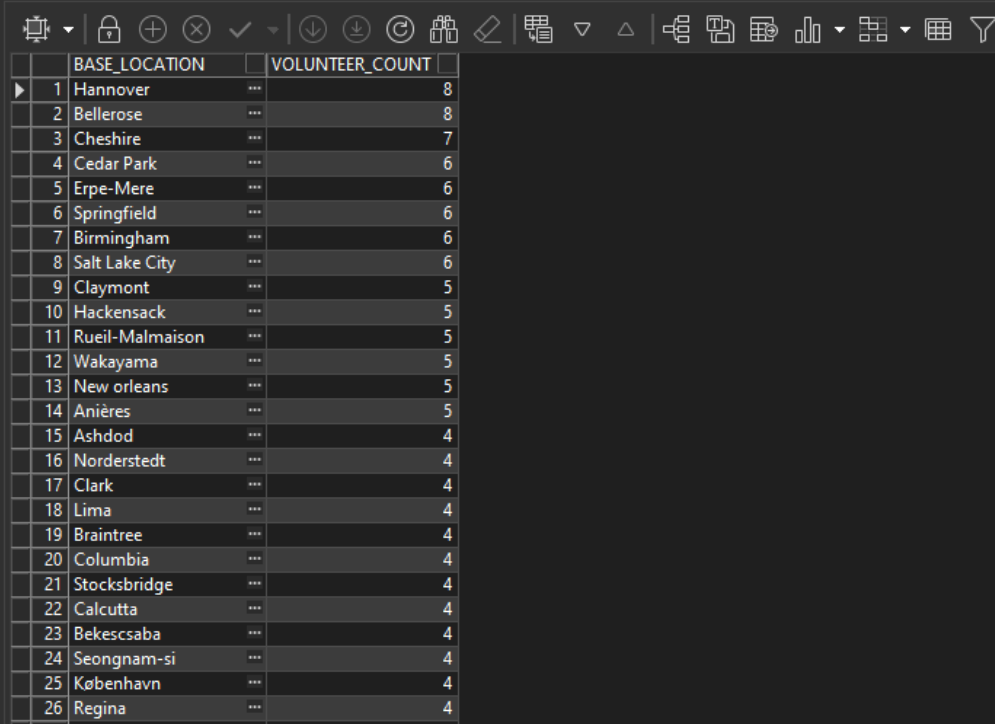
SELECT
    volunteer_name,
    phone_number,
    start_date_time,
    end_date_time,
    base_location,
    base_description
FROM
    VOLUNTEER_SHIFT_DETAILS
WHERE
    start_date_time < SYSDATE
ORDER BY
    start_date_time;
```

	VOLUNTEER_NAME	PHONE_NUMBER	START_DATE_TIME	END_DATE_TIME	BASE_LOCATION	BASE_DESCRIPTION
1	ChuckAssante	534442449	01/01/2023	25/08/2023	Washington	AMD XP 2200+ Retail box.
2	LionelBachman	538109133	01/01/2023	09/04/2023	Wien	SIS Xabre200 64MB DDR AGP 8X DVI+TV+ CRT by Power Color
3	RoddyPastore	530502678	02/01/2023	17/07/2023	København	Combo Intel P4 2.4ghz CPU + Fan(BOX) F5B800 with Hyper Threading w/
4	MorrisAkins	534501042	07/01/2023	03/06/2023	Edinburgh	Western Digital 20GB 5400RPM ATA100
5	DavisRebhorn	538895511	07/01/2023	03/06/2023	Edinburgh	Western Digital 20GB 5400RPM ATA100
6	GordLavigne	539341331	09/01/2023	21/09/2023	Pétion-ville	Combo Intel P4 1.6Ghz 478pin(box)CPU + Gigabyte GA-8IRX (DDR up to
7	RodneyBenoit	530878034	09/01/2023	21/09/2023	Pétion-ville	Combo Intel P4 1.6Ghz 478pin(box)CPU + Gigabyte GA-8IRX (DDR up to
8	DeniseLindo	535086098	10/01/2023	15/10/2023	Chambersburg	Combo AMD XP 2400+ CPU; Gigabyte GA-7VR M/B(W/DDR333 up to 3G
9	LeaKravitz	537553593	12/01/2023	21/07/2023	Frankfurt	ATI Radion 7000; 64Mb DDR AGP + CRT+ DVI+ TV
10	ErnieSpader	538389533	12/01/2023	14/09/2023	Bruxelles	Combo AMD XP 2000 Biostar M7VIK Skt A MotherBoard; KT400.
11	HookahPiven	531694270	12/01/2023	14/09/2023	Bruxelles	Combo AMD XP 2000 Biostar M7VIK Skt A MotherBoard; KT400.
12	GeoffStuermer	534147067	13/01/2023	20/12/2023	Happy Valley	ASUS A7N8X Deluxe NVIDIA Force2.
13	MiguelCondition	539138001	13/01/2023	16/12/2023	Zaandam	Logitech Smart Click 310 Camera
14	TildaGatlin	535009442	14/01/2023	23/09/2023	Murray	512MB DDR PC3200 Kingston Hyper X (KHX3200/512)
15	MaryLofgren	538428492	16/01/2023	20/05/2023	Englewood	ATI Radion 7000; 64Mb DDR AGP + CRT+ DVI+ TV
16	Johnnievon Sydow	534859721	16/01/2023	20/05/2023	Englewood	ATI Radion 7000; 64Mb DDR AGP + CRT+ DVI+ TV
17	VivicaPaxton	535740372	16/01/2023	20/05/2023	Englewood	ATI Radion 7000; 64Mb DDR AGP + CRT+ DVI+ TV
18	GaryGalecki	534740622	16/01/2023	28/02/2023	Aracruz	512MB DDR PC3200 Kingston Hyper X (KHX3200/512)
19	TamalaCurry	532829331	17/01/2023	24/02/2023	Marlboro	Black LG Electronics CD-RW 48x12x48x / DVD 16x Combo Drive.
20	JoelyChesnutt	539341331	17/01/2023	24/02/2023	Marlboro	Black LG Electronics CD-RW 48x12x48x / DVD 16x Combo Drive.

system@XE [23:38:05] 20 rows selected in 0.078 seconds (more...)

שאלתא 2 מספר מתנדבים לכל בסיס:

```
-- Count Volunteers per Base
SELECT
  base_location,
  COUNT(DISTINCT volunteer_id) AS volunteer_count
FROM
  VOLUNTEER_SHIFT_DETAILS
GROUP BY
  base_location
ORDER BY
  volunteer_count DESC;
```



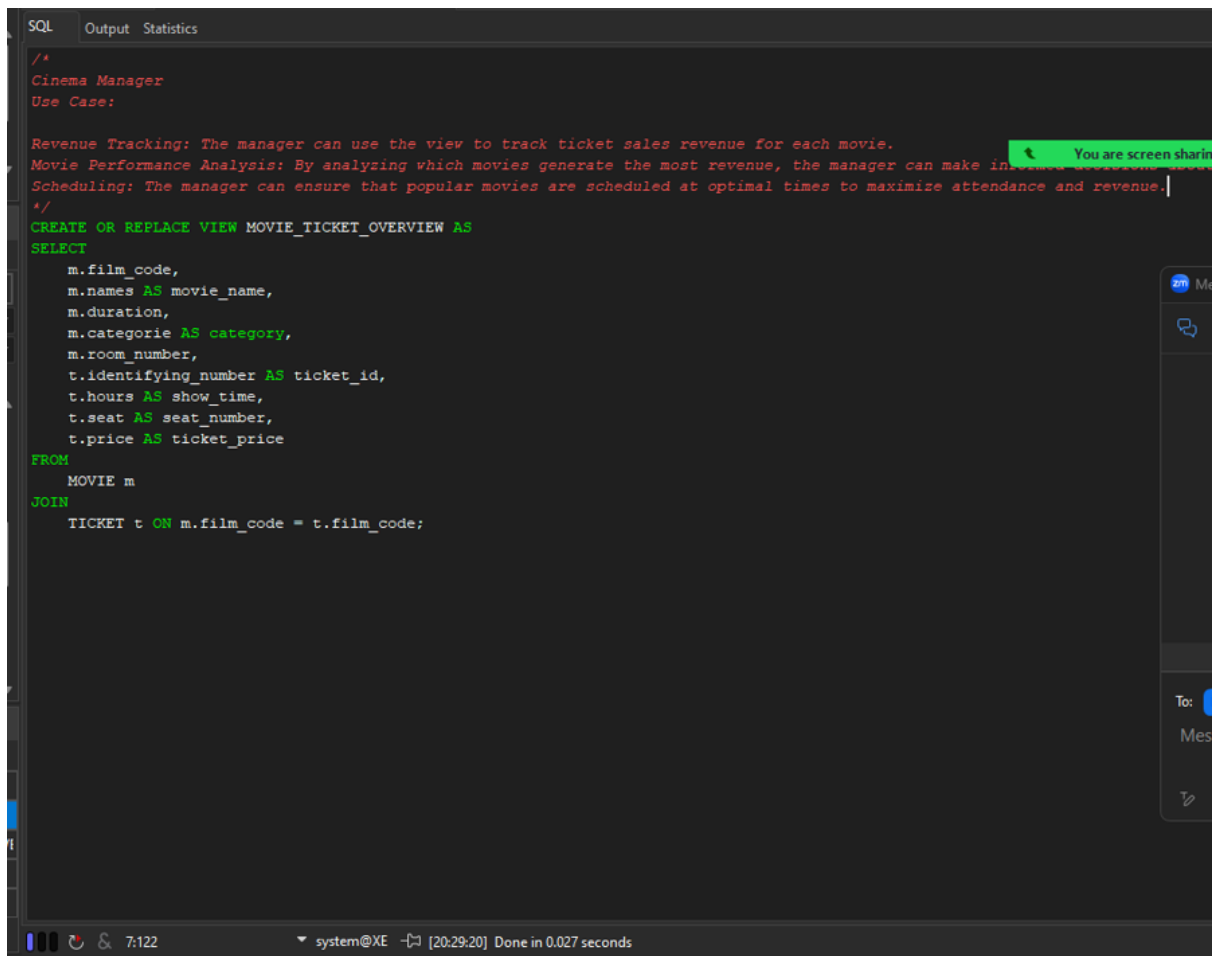
A screenshot of a database query results window. The window has a toolbar with various icons for editing and viewing data. Below the toolbar is a table with two columns: 'BASE_LOCATION' and 'VOLUNTEER_COUNT'. The table contains 26 rows of data, sorted by volunteer count in descending order. The first row is 'Hannover' with a count of 8, and the last row is 'Regina' with a count of 4. Each row has a small expand/collapse icon to its left.

	BASE_LOCATION	VOLUNTEER_COUNT
▶	1 Hannover	8
	2 Bellerose	8
	3 Cheshire	7
	4 Cedar Park	6
	5 Erpe-Mere	6
	6 Springfield	6
	7 Birmingham	6
	8 Salt Lake City	6
	9 Claymont	5
	10 Hackensack	5
	11 Rueil-Malmaison	5
	12 Wakayama	5
	13 New orleans	5
	14 Anières	5
	15 Ashdod	4
	16 Norderstedt	4
	17 Clark	4
	18 Lima	4
	19 Braintree	4
	20 Columbia	4
	21 Stocksbridge	4
	22 Calcutta	4
	23 Bekescsaba	4
	24 Seongnam-si	4
	25 København	4
	26 Regina	4

1:29

system@XE [23:46:01] 184 rows selected in 0.248 seconds

מבט מנקודת מבט שלנו:



```
SQL Output Statistics

/*
Cinema Manager
Use Case:

Revenue Tracking: The manager can use the view to track ticket sales revenue for each movie.
Movie Performance Analysis: By analyzing which movies generate the most revenue, the manager can make informed decisions.
Scheduling: The manager can ensure that popular movies are scheduled at optimal times to maximize attendance and revenue.
*/
CREATE OR REPLACE VIEW MOVIE_TICKET_OVERVIEW AS
SELECT
    m.film_code,
    m.names AS movie_name,
    m.duration,
    m.categorie AS category,
    m.room_number,
    t.identifying_number AS ticket_id,
    t.hours AS show_time,
    t.seat AS seat_number,
    t.price AS ticket_price
FROM
    MOVIE m
JOIN
    TICKET t ON m.film_code = t.film_code;
```

7:122 system@XE [20:29:20] Done in 0.027 seconds

מגישים: ישראל שלמה 315130344

אלי רוטנמר 209572676

אלמוג שפיגל 315748871

מיני פרויקט בבסיסי נתונים

```
select * from MOVIE_TICKET_OVERVIEW
```

	FILM_CODE	MOVIE_NAME	DURATION	CATEGORY	ROOM_NUMBER	TICKET_ID	SHOW_TIME	SEAT_NUMBER	TICKET_PRICE
1	1	Pierce	90	Fantasy	11	1	42799	1	10
2	2	Praga	110	Fantasy	13	2	38134	2	105
3	3	Peabo	138	Documentary	1	3	9041	3	9
4	4	Leslie	146	Animation	6	4	12534	4	319
5	5	Praga	117	Horror	15	5	61413	5	30
6	6	Tal	155	Science Fiction	14	6	57505	6	65
7	7	Madeline	95	Thriller	4	7	411	7	245
8	8	Tara	176	Action	20	8	59705	8	289
9	9	Jaime	130	Fantasy	4	9	11886	9	155
10	10	Tim	116	Horror	15	10	81104	10	169
11	11	Howie	61	Horror	18	11	5031	11	65
12	12	Joan	61	Horror	1	12	20406	12	255
13	13	Sylvester	152	Comedy	4	13	77019	13	329
14	14	Rosario	108	Science Fiction	1	14	69605	14	149
15	15	Fairuza	177	Documentary	2	15	85476	15	319
16	16	Rawlins	88	Horror	3	16	79996	16	1
17	17	Morgan	79	Romance	3	17	77613	17	89
18	18	Rene	136	Comedy	16	18	82372	18	398
19	19	Gran	155	Action	18	19	16396	19	179
20	20	Diamond	66	Animation	9	20	15790	20	240
21	21	Daniel	160	Fantasy	8	21	62676	21	99
22	22	Isaiah	84	Animation	14	22	68109	22	65
23	23	Parker	112	Drama	15	23	67136	23	48
24	24	Demi	168	Animation	11	24	14260	24	385
25	25	Brent	140	Action	2	25	26600	25	618
26	26	Suzi	75	Animation	5	26	82195	26	219
27	27	Rodney	86	Horror	19	27	83113	27	72
28	28	Eugene	70	Romance	2	28	79589	28	121
29	29	George	101	Thriller	6	29	61471	29	28
30	30	Dar	96	Documentary	5	30	30252	30	72
31	31	Jet	143	Fantasy	2	31	85682	31	199
32	32	Gwyneth	159	Documentary	9	32	38344	32	259
33	33	Jennifer	94	Romance	14	33	27836	33	69
34	34	Emma	115	Animation	5	34	46295	34	75
35	35	Carolyn	129	Comedy	8	35	15854	35	110

1:38 system@XE [20:29:41] 400 rows selected in 0.693 seconds

שאלת 1 לוקחת לכל קטגוריה את הרווח :

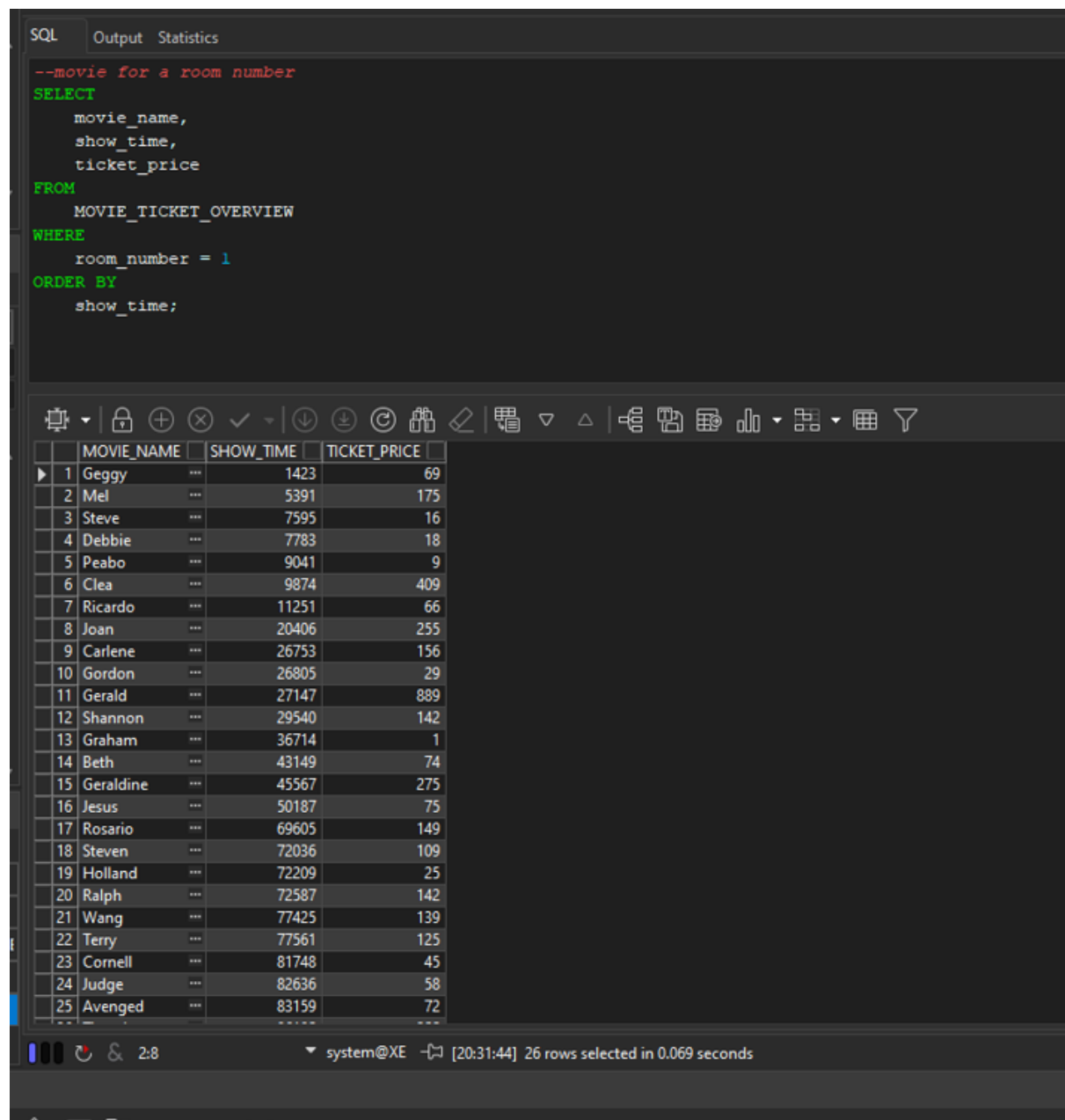
```
--movie category revenue
SELECT
    category,
    SUM(ticket_price) AS total_revenue
FROM
    MOVIE_TICKET_OVERVIEW
GROUP BY
    category
ORDER BY
    total_revenue DESC;
```

	CATEGORY	TOTAL_REVENUE
1	Fantasy	6885
2	Action	6241
3	Comedy	6234
4	Animation	5239
5	Science Fiction	5035
6	Documentary	4936
7	Horror	4603
8	Romance	4592
9	Thriller	4556
10	Drama	4032

system@XE [20:30:08] 10 rows selected in 0.072 seconds

מגישים: ישראל שלמה 315130344
אלי רוטנמר 209572676
אלמוג שפיגל 315748871

שאלת 2 לוקחת לכל סרט את הפרטים שלה לכל חדר (סך הקרנות):



The screenshot shows a SQL IDE interface with a query editor and a results pane. The query is as follows:

```
--movie for a room number
SELECT
    movie_name,
    show_time,
    ticket_price
FROM
    MOVIE_TICKET_OVERVIEW
WHERE
    room_number = 1
ORDER BY
    show_time;
```

The results pane displays a table with 26 rows and 3 columns: MOVIE_NAME, SHOW_TIME, and TICKET_PRICE. The data is sorted by show_time in ascending order.

	MOVIE_NAME	SHOW_TIME	TICKET_PRICE
1	Geggy	1423	69
2	Mel	5391	175
3	Steve	7595	16
4	Debbie	7783	18
5	Peabo	9041	9
6	Clea	9874	409
7	Ricardo	11251	66
8	Joan	20406	255
9	Carlene	26753	156
10	Gordon	26805	29
11	Gerald	27147	889
12	Shannon	29540	142
13	Graham	36714	1
14	Beth	43149	74
15	Geraldine	45567	275
16	Jesus	50187	75
17	Rosario	69605	149
18	Steven	72036	109
19	Holland	72209	25
20	Ralph	72587	142
21	Wang	77425	139
22	Terry	77561	125
23	Cornell	81748	45
24	Judge	82636	58
25	Avenged	83159	72
26			

The status bar at the bottom indicates: system@XE [20:31:44] 26 rows selected in 0.069 seconds.

מגשים: ישראל שלמה 315130344
אלי רוטנמר 209572676
אלמוג שפיגל 315748871